Maîtrise des Structures Répétitives (Boucles), Preuve de Terminaison et Invariants

Thème principal : Algorithmique et Programmation - Structures Itératives et Preuves Formelles

Synthèse exécutive : Ce cours pose les bases fondamentales de l'itération en algorithmique structurée. Il vise à éliminer l'anti-pattern du 'copier-coller' pour traiter des volumes de données variables. Le cours fournit une grille de décision claire pour choisir entre une itération bornée (boucle POUR) et une itération non bornée (TANT QUE ou RÉPÉTER... JUSQU'À). Enfin, il introduit les méthodes formelles d'assurance qualité logicielle : la preuve de terminaison grâce au 'variant de boucle' (qui garantit l'absence de boucle infinie) et la preuve de correction grâce à 'l'invariant de boucle' (qui garantit mathématiquement l'exactitude du résultat calculé).

1. Typologie des Boucles et Critères de Choix Algorithmique

Une structure répétitive (ou boucle) permet de répéter un bloc d'instructions plusieurs fois sans dupliquer le code source. On distingue deux familles fondamentales : 1. **L'itération bornée (Boucle POUR)** : À utiliser impérativement lorsque le nombre d'itérations est connu d'avance. La variable de contrôle (le compteur) est gérée automatiquement par la structure : elle est initialisée, incrémentée à chaque pas et testée par rapport à la borne supérieure. 2. **L'itération non bornée (TANT QUE vs RÉPÉTER... JUSQU'À)** : À utiliser lorsque la fin de la boucle dépend d'un évènement dynamique (une valeur saisie, la lecture d'un fichier, un seuil calculé). Le choix entre 'TANT QUE' et 'RÉPÉTER' repose sur le nombre minimal d'exécutions requises : - Dans une boucle **TANT QUE**, le test de poursuite est fait *avant* d'entrer dans le corps. Si la condition est fausse d'entrée, le bloc s'exécute 0 fois. - Dans une boucle **RÉPÉTER... JUSQU'À**, le bloc d'instructions est exécuté *d'abord*, puis la condition d'arrêt est évaluée. Le traitement est donc garanti d'être exécuté au moins 1 fois (idéal pour les contrôles de saisie ou les menus).

- Anti-pattern du Copier-Coller : Dupliquer du code le rend rigide, illisible et incapable de traiter des jeux de données dynamiques.
- Itération Bornée (POUR) : Utilisée quand le nombre total d'itérations est fixe et connu avant le premier tour de boucle.
- Itération Non Bornée à Pré-condition (TANT QUE) : La condition est évaluée avant chaque tour ; s'exécute de 0 à N fois.
- Itération Non Bornée à Post-condition (RÉPÉTER... JUSQU'À) : Le corps est exécuté au moins une fois avant le test d'arrêt ; s'exécute de 1 à N fois.

Cas concret enseignant : *L'analogie du coureur sur la piste d'athlétisme : Si l'entraîneur dit 'Fais 10 tours de piste', c'est une boucle POUR. S'il dit 'Cours tant qu'il ne pleut pas', c'est un TANT QUE (s'il pleut déjà au vestiaire, le coureur ne commence même pas = 0 tour). S'il dit 'Fais un tour et arrête-toi dès que tu es essoufflé', c'est un RÉPÉTER... JUSQU'À (le coureur effectue obligatoirement le premier tour).*

2. Preuve Formelle de Terminaison : Le Variant de Boucle

Pour démontrer formellement qu'une boucle non bornée s'arrête (preuve de terminaison), il faut identifier un 'variant de boucle'. Un variant V est une fonction entière qui exprime une quantité calculée sur les variables de la boucle. La preuve mathématique repose sur la propriété du bon ordre des entiers naturels : il n'existe pas de suite d'entiers strictement décroissante et minorée qui soit infinie. Pour prouver qu'un algorithme termine, il faut démontrer deux propriétés : 1. **Décroissance stricte** : À chaque itération k, la valeur V_{k+1} est strictement inférieure à V_k. 2. **Minoration** : Il existe un entier constant M (souvent 0) tel que dès que $V \le M$, la condition de boucle devient fausse. Si ces deux conditions sont satisfaites, la terminaison de la boucle est scientifiquement garantie.

- Boucle Infinie : Dysfonctionnement critique provoquant le gel de l'application si la condition de sortie n'est jamais atteinte.

• Variant de Boucle : Grandeur entière relative à l'état du programme mesurant la 'distance' jusqu'à l'arrêt.
• Décroissance Stricte : Le variant doit diminuer d'au moins 1 entier à chaque tour de boucle (V_{k+1} < V_k).
• Minorant : Valeur plancher (généralement 0) en dessous de laquelle le variant ne peut pas descendre.

Cas concret enseignant : *La métaphore du réservoir d'essence d'une voiture : À chaque kilomètre parcouru (itération), le volume d'essence dans le réservoir (le variant) diminue obligatoirement d'une quantité stricte. Comme le réservoir ne peut pas descendre sous 0 litre (minorant), la voiture finira obligatoirement par s'immobiliser. Si la jauge ne baissait pas à chaque kilomètre, la voiture roulerait à l'infini : c'est le bug de la boucle infinie.*

3. Preuve Formelle de Correction : L'Invariant de Boucle

Savoir qu'une boucle s'arrête ne suffit pas : il faut prouver qu'elle calcule le bon résultat. C'est le rôle de l'invariant de boucle, qui fonctionne exactement comme un raisonnement par récurrence : 1. **Initialisation** : On prouve que l'invariant P est vrai juste avant la première itération ($k=0$). 2. **Conservation (Hérédité)** : On suppose que P est vrai au début d'une itération quelconque k. On démontre qu'après l'exécution des instructions du corps de la boucle, P demeure vrai pour le tour $k+1$. 3. **Conclusion / Correction** : Quand la boucle termine, la condition de maintien devient fausse (Not C) et l'invariant P est toujours vrai. La conjonction logique $(P \land \neg C)$ permet de déduire mathématiquement que le résultat produit correspond à la spécification de l'algorithme.

• Invariant de Boucle : Assertion logique (propriété) qui reste VRAIE avant, pendant et après chaque itération.
• Initialisation : Démonstration que l'invariant est vrai dès l'entrée dans la boucle.
• Conservation (Hérédité) : Démonstration que si l'invariant est vrai au début du tour k, il le reste à la fin du tour k.
• Correction (Conclusion) : Combinaison de l'invariant final et du test d'arrêt prouvant la validité du résultat.

Cas concret enseignant : *L'exemple du livreur de journaux : Au départ de sa tournée, l'égalité 'Journaux distribués + Journaux restants dans le sac = Total initial' est vraie. À chaque maison servie, les deux quantités changent, mais leur somme reste rigoureusement la même (invariant). Quand le sac est vide à la fin de la tournée (condition de sortie), le nombre de journaux distribués est obligatoirement égal au total initial.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Est-ce qu'on peut toujours remplacer une boucle POUR par une boucle TANT QUE ?
Réponse du professeur : Oui, sur le plan logique, une boucle POUR peut systématiquement être réécrite avec une boucle TANT QUE. Il suffit d'initialiser le compteur avant la boucle, de placer le test de borne dans la condition TANT QUE, et d'incrémenter explicitement le compteur à la toute fin du corps. Cependant, si le nombre de tours est connu à l'avance, la boucle POUR est à privilégier car elle gère le compteur de manière sûre et évite l'oubli de l'incrémentation.

Question étudiant : Si j'utilise une boucle TANT QUE avec la condition (i < N), est-ce que le corps s'exécute si i est déjà égal à N dès le début ?
Réponse du professeur : Non. Dans une boucle TANT QUE, la condition est évaluée AVANT de rentrer dans la boucle. Si i est égal à N dès le départ, la condition (i < N) est immédiatement évaluée à FAUX. Le corps de la boucle est donc complètement ignoré et exécuté exactement 0 fois.

■ Exercice & Démonstration d'Application

Énoncé : Écrire un algorithme en pseudo-code permettant de calculer la somme des N premiers entiers pairs stricts (par exemple pour N=3, sommer 2 + 4 + 6 = 12). Justifier le choix de la boucle, rédiger le code algorithmique et formaliser le variant de boucle garantissant la terminaison.

Résolution pas à pas :
Étape 1 : Choix de la boucle. La valeur N est connue dès l'entrée en paramètre ; le nombre d'itérations est donc déterminé et borné à N. La boucle 'POUR' est la structure la plus adaptée et la plus sécurisée. Étape 2 : Initialisation des variables. - somme : entier servant d'accumulateur, initialisé à 0. - terme : entier représentant le prochain terme pair à ajouter, initialisé à 2. - i : variable de contrôle de boucle allant de 1 à N. Étape 3 : Écriture du

pseudo-code. Algorithme SommePairs Variables : N, i, somme, terme : Entiers Début Lire(N) somme <- 0 terme <- 2 Pour i allant de 1 à N par pas de 1 Faire somme <- somme + terme terme <- terme + 2 FinPour Afficher(somme) Fin Étape 4 : Démonstration du variant de boucle. Si l'on traduit cette itération sous forme de boucle tant que contrôlée par le compteur i, la quantité $V = (N - i + 1)$ est un entier naturel. À chaque tour de boucle, i est incrémenté de 1, donc V diminue strictement de 1 ($V_{k+1} = V_k - 1$). Le minorant est 0 (obtenu quand $i = N + 1$). Dès que $V \le 0$, la condition de boucle devient fausse, prouvant la terminaison en exactement N itérations.

Résultat : Pour N = 3 : - Tour 1 (i=1) : somme = 0 + 2 = 2 ; terme devient 4. - Tour 2 (i=2) : somme = 2 + 4 = 6 ; terme devient 6. - Tour 3 (i=3) : somme = 6 + 6 = 12 ; terme devient 8. Sortie de boucle. La valeur affichée est 12. L'algorithme se termine de manière scientifiquement prouvée et produit le résultat correct.

■ Auto-Évaluation Flash d'Examen

Q : Quelle est la différence fondamentale en mémoire et à l'exécution entre une boucle TANT QUE et une boucle RÉPÉTER... JUSQU'À ?

R : La boucle TANT QUE évalue sa condition de maintien en pré-condition (avant le corps), permettant entre 0 et N exécutions. La boucle RÉPÉTER... JUSQU'À évalue sa condition en post-condition (après le corps), garantissant entre 1 et N exécutions.

Q : Quelles sont les deux propriétés mathématiques indispensables qu'une fonction entière doit vérifier pour constituer un variant de boucle valide ?

R : 1) Elle doit décroître strictement d'au moins 1 entier à chaque itération ($V_{k+1} < V_k$). 2) Elle doit être minorée par une constante (généralement 0) en dessous de laquelle la boucle s'arrête obligatoirement.

Q : Quel oubli classique dans l'écriture d'une boucle TANT QUE entraîne une boucle infinie et fait perdre l'intégralité des points en DS d'algorithmique ?

R : L'oubli de la mise à jour (incrémentation ou modification) de la variable de contrôle à l'intérieur du corps de la boucle. Sans cette mise à jour, la condition reste éternellement vraie, créant une boucle infinie.